Improving Vision Transformer's Time Perfomace via Linear Attention and Token Pruning

Chi-Van Vong and Minh-Quan Pham Nguyen

University of Science, Vietnam National University Ho Chi Minh City, Vietnam
{vcvan2424,pnmquan2414}@apcs.fitus.edu.vn

Abstract. **Keywords:** Vision Transformer, Linear Attention, Reinforcement Learning

1 Introduction

In recent years, **Transformer** [15] has become a powerful foundational architecture for many models. **Vision Transformers** [2] one of the leading architectures in Computer Vision, have made significant breakthroughs in object detection, image classification, and semantic segmentation. However, softmax attention, the core mechanism of Transformer, has a high computational complexity of $O(n^2)$. To address the issue, many works attempted to group vision tokens. Although this method reduces the complexity, Transformer loses its ability to perceive information globally. Other works replace softmax attention with **linear attention** which help reduce the quadratic complexity. However, linear attention inability to model complex information yields unsatisfactory results, due to its low-rank nature and magnitude negligence. Many works have attempted to improve linear attention by tackling its low-rank nature [3]. Linear attention replaces softmax with kernal feature maps which causes the loss of sharpness. This mean that it can not be as selective as softmax attention which can place most of its attention weights on a few relevant tokens. To address this, we apply a RL **weighting** mechanism for each token, and extended to **token pruning** [12,9] which help refine the input for linear attention. With this approach, we aim to speed up the training and inference time of the model without sacrificing its performance.

2 Related Work

2.1 Vision Transformer

Transformer [15] model is initially created for NLP tasks and achieved remarkable success. However, they did not have much impact on computer vision until Alexey Dosovitskiy et al. [12] introduced the Vision Transformer (ViT), which splits the image into many small patches as tokens feed to the model. Although this approach demonstrated strong performance on many Computer Vision tasks, its quadratic complexity due to self-attention making it computationally expensive to process high resolution images. To address this, Microsoft Research proposed Swin Transformer [8], which restricts attention to local windows, inspiring subsequent window-based variants. Other methods reduce attention cost by compressing keys or values using linear projections, strided convolutions, pooling, or patch clustering. Additionally, some variants explore alternative designs such as channel attention and softmax-free attention mechanisms.

2.2 Linear Attention

Linear Attention [14] fixes the quadratic complexity of softmax attention by using kernel functions to approximate the $Sim()$ operator which reduce the complexity to linear. Given an input sequence of N tokens, standard self-attention computes

$$A = \text{softmax}(\frac{QK^T}{\sqrt{d}})V \tag{1}$$

where $Q, K, V \in \mathbb{R}^{N\times d}$ are the query, key, and value matrices and d is the feature dimension.

To reduce the complexity of softmax attention, linear attention replaces softmax operation with a feature map $\phi(\cdot)$

$$A = \phi(Q)\phi(K)^T V \tag{2}$$

where $\phi(\cdot)$ is a kernel feature mapping applied elementwise. Due to matrix rank constraints,

$$\text{rank}(A) \leq d. \tag{3}$$

When the number of tokens N is significantly larger than d, the attention mechanism must compress the interactions among many tokens into a limited representation. This compression can reduce the model's ability to capture fine-grained relationships among tokens, especially in high-resolution vision tasks. Thus its performance is still behind softmax. Many works try to tackle this issue [1,5,6,7,10,14]. FLattenTransformer [5] introduces a focused function to enhance the ability of linear attention to concentrate on relevant information, while MLLA [6] establishes a link between linear attention and the Mamba model [4]. Many other approaches either rely on kernel-based approximations of the softmax function

or integrate depthwise convolution to improve feature diversity. Recently, Qihang Fan et al. [3] addresses a fundamental limitation of linear attention by identifying the low-rank nature of its feature representations as a major cause of performance degradation. To overcome this, the paper proposes Rank-Augmented Linear Attention, which enriches both the key-value buffer and output features to raise their effective rank.

2.3 Token Pruning

ViT suffers from high computational cost propotional to the number of input tokens, hence many attempts have been made to reduce the redundant tokens while keeping the high performance. Traditional dynamic methods like DynamicViT [12] and TokenLearner [13] adaptively drop or condense tokens. Adaptive pruning technique such as AdaViT [11] use attention metrics or learned masks to select important tokens. Recent methods introduce the use of RL to learn the pruning policies. Specifically, Chenglong Lu et al. [9] formulates token pruning as a sequential decision making problem with multi-agent RL, where each of the visual token is consider an agent.

3 Method

We adopt the method from RALA for solving the low rank problem, and replace the weighting component with an RL weighting framework which is a continuous, stochastic Actor-Critic module. We aim to add a budget constraint, and layer wise information into the weighting to improve the linear attention speed and performance.

3.1 Architecture Overview

An input image is tokenized into N patch embeddings and partitioned into T sequential chunks of size C. A linear-attention ViT backbone processes each chunk, projecting it into local queries Q_t, keys K_t, and values V_t and maintaining a recurrent memory (S_t, Z_t) in place of a full attention matrix. At each step t, an Actor-Critic module observes the causal state local features, compressed memory, and remaining budget—and emits per-token weights $w_t \in (0,1)^C$. These weights gate the memory update $S_t = S_{t-1} + (w_t \odot \phi(K_t))^\top V_t$. After all T chunks, the final state (S_T, Z_T) is consumed by a task head (classifier or detection decoder), which produces the terminal reward.

Training follows a two-phase curriculum. **Phase 1** bootstraps the routing module from Fisher Information saliency targets, avoiding the policy collapse that arises when RL is applied cold to a high-dimensional visual state. **Phase 2** fine-tunes the full system end-to-end on the downstream task via PPO with Generalized Advantage Estimation. During inference, stochastic sampling is disabled, the Actor's deterministic outputs are hard-thresholded at τ, physically pruning low-weight tokens from the GPU buffer and yielding exact FLOP reductions.

3.2 Chunkwise Formulation of Linear Attention

Standard linear attention leverages the associative property to reduce complexity to $\mathcal{O}(N)$. The output for the i-th query is typically computed using a global normalization term:

$$O_i = \frac{\phi(Q_i) \sum_{j=1}^{N} \phi(K_j)^\top V_j}{\phi(Q_i) \sum_{j=1}^{N} \phi(K_j)^\top} \tag{4}$$

To enable sequential importance assignment, we reformulate this into a causal, chunkwise process. The input sequence of N tokens is partitioned into T non-overlapping sequential chunks of size C. At each step $t \in \{1, \ldots, T\}$, the backbone processes the current chunk into local Queries (Q_t), Keys (K_t), and Values (V_t).

Rather than computing a full sequence matrix, we maintain two recurrent memory representations to capture global context. The numerator state $S_t \in \mathbb{R}^{D \times D}$ accumulates the cross-covariance of keys and values, while the denominator state $Z_t \in \mathbb{R}^{D \times 1}$ tracks the normalization factor:

$$S_t = S_{t-1} + \phi(K_t)^\top V_t \tag{5}$$

$$Z_t = Z_{t-1} + \sum_{i \in \text{chunk } t} \phi(K_{i,t})^\top \tag{6}$$

The attention output for the current chunk t is then computed as:

$$O_t = \frac{\phi(Q_t) S_t}{\phi(Q_t) Z_t} \tag{7}$$

3.3 Markov Decision Process Formulation

By reformulating linear attention into a recurrent, chunkwise execution, we isolate the processing of each image segment into a discrete-time Markov Decision Process (MDP). This structure allows a routing agent to sequentially determine the importance of tokens based on local information, accumulated global memory, and remaining computational resources.

State Space ($\mathcal{S}$) At each chunk step t, the agent observes a causal state s_t defined as:

$$s_t = [K_t \parallel \text{Proj}(S_{t-1}) \parallel \text{Proj}(Z_{t-1}) \parallel B_t] \tag{8}$$

The components are:

- **Local Features ($K_t \in \mathbb{R}^{C \times D}$):** Visual features of the current chunk.
- **Memory Projection ($\text{Proj}(S_{t-1}) \in \mathbb{R}^D$):** A learned projection of the accumulated memory matrix $S_{t-1} \in \mathbb{R}^{D \times D}$.
- **Normalization Projection ($\text{Proj}(Z_{t-1}) \in \mathbb{R}^D$):** A projection of the normalization accumulator Z_{t-1}, encoding the effective mass of stored tokens.
- **Dynamic Budget ($B_t \in [0,1]$):** Remaining computational budget.

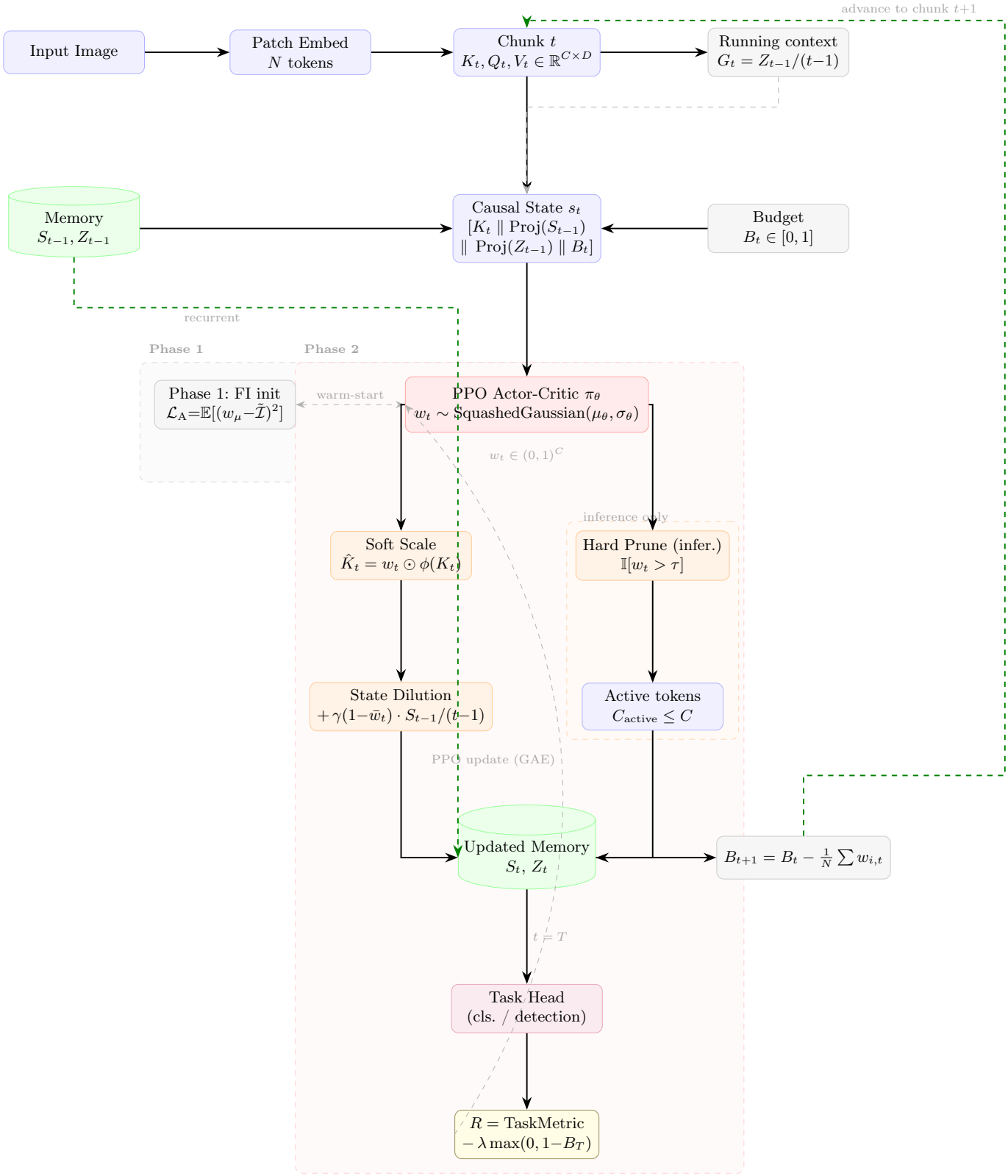

Fig. 1. Architecture overview. An input image is tokenised into N patches and processed in T sequential chunks. At each step t, the PPO Actor-Critic observes the causal state s_t encoding local features, projections of the recurrent memory (S_{t-1}, Z_{t-1}), and the remaining budget B_t and emits per-token weights w_t. These soft-scale token contributions during training; at inference, weights below threshold τ are hard-pruned for exact FLOP reduction. State Dilution replenishes the memory with the historical mean $S_{t-1}/(t-1)$ proportional to suppression $(1 - \bar{w}_t)$. After T chunks, the final memory (S_T, Z_T) feeds a task head and a terminal reward drives PPO via GAE. Phase 1 bootstraps the Actor from Fisher Information saliency targets before Phase 2 end-to-end fine-tuning.

Action Space ($\mathcal{A}$) For each chunk of size C, the Actor π_θ outputs:

$$a_t = \{w_{1,t}, w_{2,t}, \ldots, w_{C,t}\}, \quad w_{i,t} \in (0,1) \tag{9}$$

Transition Dynamics The transition to s_{t+1} is governed by deterministic updates of the memory and normalization accumulators.

The memory update is:

$$S_t = S_{t-1} + \big((w_t \odot \phi(K_t))^\top V_t\big) \tag{10}$$

The normalization term evolves as:

$$Z_t = Z_{t-1} + \sum_{i=1}^{C} w_{i,t}\, \phi(K_{i,t})^\top, \tag{11}$$

where $\phi(\cdot)$ is the feature map associated with linear attention, and $\odot$ denotes element-wise scaling of token features by their corresponding weights.

The budget follows:

$$B_{t+1} = B_t - \frac{1}{N}\sum_{i=1}^{C} w_{i,t} \tag{12}$$

Reward Function ($\mathcal{R}$) A terminal reward R is provided at $t = T$:

$$R = \text{TaskMetric} - \lambda_{\text{penalty}} \cdot \max(0, 1 - B_T) \tag{13}$$

where:

- **TaskMetric:** Task-specific performance (e.g., mAP or accuracy).
- **Budget Penalty:** Penalizes exceeding the allocated token budget.

This formulation encourages the agent to preserve informative tokens while respecting computational constraints.

3.4 Squashed Gaussian Actor-Critic Network

To provide the flexibility to down-weight or entirely omit uninformative chunks, we model the routing policy $\pi_\theta(a_t \mid s_t)$ as a Squashed Gaussian distribution. This independent weighting avoids the mathematical instability of discrete gates and the competitive constraints of a global Softmax operation, allowing the network to allocate exact importance independent of the surrounding context.

The Actor network computes the parameters for an unbounded Normal distribution. To ensure the standard deviation remains positive and numerically stable, it is passed through a Softplus activation with a small ϵ floor:

$$\mu_\theta = \text{MLP}_\mu(s_t), \quad \sigma_\theta = \text{Softplus}(\text{MLP}_\sigma(s_t)) + \epsilon$$

During stochastic training, actions are sampled using the reparameterization trick, allowing gradients to flow back through the stochastic node. The unbounded sample x is then squashed via a hyperbolic tangent and shifted to the valid continuous weighting space of $(0,1)$:

$$w_t = \frac{\tanh(x) + 1}{2}, \quad x \sim \mathcal{N}(\mu_\theta, \sigma_\theta^2)$$

By decoupling the weights from a global sum, the Actor can assign near-zero weights to every token in a redundant chunk, effectively "muting" its contribution to the global memory S_t. However, applying the nonlinear Tanh transformation changes the probability density of the sampled action. To maintain mathematical integrity for Proximal Policy Optimization (PPO), the log-probability of the action must be corrected using the Jacobian trace of the Tanh transformation:

$$\log \pi(w_t) = \log p(x) - \sum \log(1 - \tanh^2(x) + \epsilon)$$

Concurrently, a Value network (Critic) $V_\phi(s_t)$ evaluates the causal state to estimate the expected episodic return, which serves as a baseline to stabilize the subsequent RL updates.

3.5 Pretraining: Scale-Aligned Information Distillation

Initializing an RL agent from scratch in a massive parameter space often leads to representation collapse, as the agent initially acts randomly and destroys the transformer's memory state. To circumvent this, Phase 1 initializes the Actor using empirical Fisher Information ($\mathcal{I}_i$) as a saliency oracle.

After warming up the backbone on a dense classification task, we compute the squared gradients of the cross-entropy loss with respect to the token features to approximate their Fisher Information. These scores reflect the true informational utility of each token. To prevent scale shift during the transfer to the RL phase, we apply spatial smoothing and normalize these scores to a strict $\tilde{\mathcal{I}}_i \in [0,1]$ range.

The Actor and Critic are then trained via supervised learning. The Actor minimizes the Mean Squared Error between its deterministic mean prediction w_μ and the Fisher utility, while the Critic learns to predict the total saliency of the chunk:

$$\mathcal{L}_{\text{Actor}} = \mathbb{E}\left[\left(w_\mu(s_t) - \tilde{\mathcal{I}}_t\right)^2\right]$$

$$\mathcal{L}_{\text{Critic}} = \mathbb{E}\left[\left(V_\phi(s_t) - \sum_{i \in t} \tilde{\mathcal{I}}_i\right)^2\right]$$

3.6 Finetuning: Task-Specific Reward Optimization

In Phase 2, the model is transitioned to the downstream task and optimized via PPO. The environment dynamically enforces a decreasing budget B_t, forcing the Actor to plan its token allocation temporally. Aggressive down-weighting can cause the recurrent memory to stagnate, leading to catastrophic forgetting of accumulated visual context. To mitigate this, the memory is augmented at each step by a fraction of its own historical average, scaled by the degree of suppression $(1 - \bar{w}_t)$:

$$S_t = S_{t-1} + (w_t \odot \phi(K_t))^\top V_t + \gamma \cdot (1 - \bar{w}_t) \cdot \frac{S_{t-1}}{t-1} \quad (14)$$

where the dilution term is defined as zero for $t = 1$. When the agent fully suppresses a chunk ($\bar{w}_t \to 0$), the memory is topped up with the mean KV contribution accumulated so far, preventing stagnation. When all tokens are retained ($\bar{w}_t \to 1$), the term vanishes and the update reduces to standard linear attention.

Using Generalized Advantage Estimation ($\hat{A}_t$) derived from the Critic's value predictions, the Actor is updated via the clipped PPO surrogate objective. To guarantee sufficient exploration of the continuous weight space and prevent premature convergence, we inject an entropy bonus:

$$\begin{aligned}\mathcal{L}_{\text{PPO}} = &- \mathbb{E}\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1 \pm \epsilon)\hat{A}_t\Big)\Big] \\ &- \lambda_{\text{ent}}\, H(\pi_\theta)\end{aligned} \quad (15)$$

where

$$H(\pi_\theta) = \frac{1}{2}\log(2\pi e\, \sigma_\theta^2) + \mathbb{E}\left[\sum_i \log(1 - \tanh^2(x_i) + \epsilon)\right] \quad (16)$$

is the entropy of the squashed Gaussian policy.

During inference, stochastic sampling is disabled. The Actor deterministically outputs a continuous weight w_{infer}. A hard threshold τ enables physical token purging, stripping uninformative tokens from the GPU memory buffer to yield exact FLOP reductions. Crucially, surviving tokens retain their continuous scaling factors w_{infer} in the linear attention update, bypassing the quantization shock typically associated with discrete gating mechanisms.

4 Experiments

4.1 Experimental Design

Our evaluation includes two primary computer vision tasks: **image classification** and **object detection**. We focus on measuring the trade-off between model accuracy and computational efficiency (both training and inference) while conducting ablation studies to isolate the impact of each architectural component.

Task Specifications

Image Classification We use the **ImageNet-100** dataset, a standard benchmark containing over 100 thousand images across 200 categories. Performance is measured using **Top-1 Accuracy**, which denotes the percentage of instances where the model's highest-probability prediction matches the ground-truth label.

Object Detection For localization tasks, we use **COCO** 2017 which is widely recognized for its complexity and variety of object scales. We report **Average Precision (AP)**, a metric that accounts for both the precision and recall of the predicted bounding boxes at various Intersection over Union (IoU) thresholds.

4.2 Implementation Environment

The framework is implemented in **Python** using **PyTorch**. All experiments, including training and inference profiling, are conducted on a local computer with a single NVIDIA RTX 5060Ti 16GB.

Cost	Model	Params(M)	FLOPs(G)	Top1-acc(%)
2.5G ≀	RALA			
	RMT-T			
	Ours			

Table 1. Classification results of different general backbones on ImageNet-100

5 Results

- object detection: RALA, Ours, RMT-T
- image classification: RALA, Ours, MLLA-T
- inference speed
- ablation study: softmax, linear, non RL

6 Discussion

Need to conduct ablation study.

7 Conclusion

Cost	Model	Params(M)	FLOPs(G)	Top1-acc(%)
2.5G ≀	RALA			
	MLLA-T			
	Ours			

Table 2. Object detection results of different general backbones on COCO 2017

References

1. Cai, H., Li, J., Hu, M., Gan, C., Han, S.: Efficientvit: Multi-scale linear attention for high-resolution dense prediction. In: ICCV (2023), https://arxiv.org/abs/2205.14756
2. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al.: An image is worth 16x16 words: Transformers for image recognition at scale. In: ICLR (2020), https://arxiv.org/abs/2010.11929
3. Fan, Q., Huang, H., He, R.: Breaking the low-rank dilemma of linear attention. In: CVPR (2025), https://arxiv.org/abs/2411.07635
4. Gu, A., Dao, T.: Mamba: Linear-time sequence modeling with selective state spaces (2024), https://arxiv.org/abs/2312.00752
5. Han, D., Pan, X., Han, Y., Song, S., Huang, G.: Flatten transformer: Vision transformer using focused linear attention. In: ICCV (2023), https://arxiv.org/abs/2308.00442
6. Han, D., Wang, Z., Xia, Z., Han, Y., Pu, Y., Ge, C., Song, J., Song, S., Zheng, B., Huang, G.: Demystify mamba in vision: A linear attention perspective. In: NeurIPS (2024), https://arxiv.org/abs/2405.16605
7. Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F.: Transformers are rnns: Fast autoregressive transformers with linear attention. In: ICML (2020), https://arxiv.org/abs/2006.16236
8. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B.: Swin transformer: Hierarchical vision transformer using shifted windows. In: ICCV (2021), https://arxiv.org/abs/2103.14030
9. Lu, C., Liang, S., Wang, X., Wang, W.: Reinforcement learning-based token pruning in vision transformers: A markov game approach. In: ICME (2025), https://arxiv.org/abs/2503.23459
10. Lu, J., Zhang, J., Zhu, X., Feng, J., Xiang, T., Zhang, L.: Softmax-free linear transformers. In: IJCV (2024), https://arxiv.org/abs/2207.03341
11. Meng, L., Li, H., Chen, B.C., Lan, S., Wu, Z., Jiang, Y.G., Lim, S.N.: Adavit: Adaptive vision transformers for efficient image recognition. In: CVPR (2022), https://arxiv.org/abs/2111.15668
12. Rao, Y., Zhao, W., Liu, B., Lu, J., Zhou, J., Hsieh, C.J.: Dynamicvit: Efficient vision transformers with dynamic token sparsification. In: NeurIPS (2021), https://arxiv.org/abs/2106.02034
13. Ryoo, M.S., Piergiovanni, A., Arnab, A., Dehghani, M., Angelova, A.: Tokenlearner: What can 8 learned tokens do for images and videos? arXiv preprint arXiv:2106.11297 (2021), https://arxiv.org/abs/2106.11297
14. Shen, Z., Zhang, M., Zhao, H., Yi, S., Li, H.: Efficient attention: Attention with linear complexities. In: WACV (2021), https://arxiv.org/abs/1812.01243
15. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: NeurIPS (2017), https://arxiv.org/abs/1706.03762